1. Environment Setup

```
#23070521013 Amritansh Singh
import sys

# Install required libraries
!{sys.executable} -m pip install tensorflow matplotlib scikit-learn
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.13/dist-packages (2.20.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from ter
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.13/di
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.13/dist-packages (frc
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.13/dist-packages (from t
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: packaging in /usr/local/lib/python3.13/dist-packages (from tensorfl
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.13/dist-packages (from t
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.13/dist-packages (frc
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from tensorf
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.13/dist-packages (from tensor
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from t
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.13/dist-packages (from tens
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.13/dist-packages (frc
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.13/dist-packages (frc
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.13/dist-packages (from tens
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from tens
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.13/dist-packages (from tensc
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from m
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matpl
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from matplotl
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from m
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from sciki
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scik
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (fr
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: rich in /usr/local/lib/python3.13/dist-packages (from keras>=3.10.0
Requirement already satisfied: namex in /usr/local/lib/python3.13/dist-packages (from keras>=3.10.
Requirement already satisfied: optree in /usr/local/lib/python3.13/dist-packages (from keras>=3.10
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from reque
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.13/dist-packages (from te
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.13/
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from te
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.13/dist-packages (from
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (f
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdow
```

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import VGG16, ResNet50, MobileNetV2
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import os
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

---

2. Data Loading and Preparation

```
# This cell sets up a dummy dataset structure for demonstration purposes.
# In a real scenario, replace this with your actual dog breed dataset.

print("Please upload your dog breed dataset or specify a path to an existing dataset.")
print("For example, you can use the 'Stanford Dogs Dataset' or 'Dogs vs Cats' dataset.")
print("Make sure your dataset is organized into subdirectories by breed for easy loading.")

dataset_path = 'dog_breed_dataset'

# Create dummy directories and files if they don't exist
if not os.path.exists(dataset_path):
    os.makedirs(os.path.join(dataset_path, 'train', 'breed1'))
    os.makedirs(os.path.join(dataset_path, 'train', 'breed2'))
    os.makedirs(os.path.join(dataset_path, 'validation', 'breed1'))
    os.makedirs(os.path.join(dataset_path, 'validation', 'breed2'))
    os.makedirs(os.path.join(dataset_path, 'test', 'breed1'))
    os.makedirs(os.path.join(dataset_path, 'test', 'breed2'))

    from PIL import Image
    img = Image.new('RGB', (224, 224), color = 'red')
    img.save(os.path.join(dataset_path, 'train', 'breed1', 'dog1.jpg'))
    img.save(os.path.join(dataset_path, 'train', 'breed2', 'dog2.jpg'))
    img.save(os.path.join(dataset_path, 'validation', 'breed1', 'dog3.jpg'))
    img.save(os.path.join(dataset_path, 'test', 'breed2', 'dog4.jpg'))
    print(f"Dummy dataset structure created at '{dataset_path}' for demonstration.")
else:
    print(f"Dataset path '{dataset_path}' already exists.")

IMAGE_SIZE = (224, 224)
BATCH_SIZE = 32

# Data augmentation and preprocessing
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

validation_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

# Load datasets using flow_from_directory
train_generator = train_datagen.flow_from_directory(
    os.path.join(dataset_path, 'train'),
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
```

```
)

validation_generator = validation_datagen.flow_from_directory(
    os.path.join(dataset_path, 'validation'),
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)

test_generator = test_datagen.flow_from_directory(
    os.path.join(dataset_path, 'test'),
    target_size=IMAGE_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)

print("Data generators created successfully.")
```

```
Please upload your dog breed dataset or specify a path to an existing dataset.
For example, you can use the 'Stanford Dogs Dataset' or 'Dogs vs Cats' dataset.
Make sure your dataset is organized into subdirectories by breed for easy loading.
Dataset path 'dog_breed_dataset' already exists.
Found 2 images belonging to 2 classes.
Found 1 images belonging to 2 classes.
Found 1 images belonging to 2 classes.
Data generators created successfully.
```

---

3. Model Selection and Loading

```
# Determine the number of classes from the training generator
num_classes = train_generator.num_classes
print(f"Number of classes: {num_classes}")

def build_transfer_learning_model(base_model_name, input_shape, num_classes):
    """Builds a transfer learning model using a specified pre-trained base model."""

    if base_model_name == 'VGG16':
        base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)
    elif base_model_name == 'ResNet50':
        base_model = ResNet50(weights='imagenet', include_top=False, input_shape=input_shape)
    elif base_model_name == 'MobileNetV2':
        base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=input_shape)
    else:
        raise ValueError("Unsupported base model name. Choose from 'VGG16', 'ResNet50', 'MobileNe

    # Freeze the base model layers
    base_model.trainable = False

    # Add custom layers on top of the base model
    x = base_model.output
    x = Flatten()(x)
    x = Dense(512, activation='relu')(x)
    x = Dropout(0.5)(x)
    predictions = Dense(num_classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=predictions)

    # Compile the model
    model.compile(optimizer=Adam(learning_rate=0.0001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
```

```
    return model

# Choose a base model and build the full model
# Let's start with VGG16 as it's a common and effective choice for transfer learning.
model_name = 'VGG16'
input_shape = IMAGE_SIZE + (3,) # Add channel dimension

model = build_transfer_learning_model(model_name, input_shape, num_classes)

model.summary()
print(f"Model built using {model_name} as the base.")
```

```
Number of classes: 2
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_wei
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "functional"
```

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 512)	12,845,568
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 2)	1,026

```
Total params: 27,561,282 (105.14 MB)
Trainable params: 12,846,594 (49.01 MB)
```

4. Model Training (Fine-tuning)

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

# Define callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
model_checkpoint = ModelCheckpoint('best_dog_breed_model.keras', save_best_only=True, monitor='va

# Train the model
history = model.fit(
    train_generator,
    epochs=20, # You might need to adjust this based on dataset size and convergence
    validation_data=validation_generator,
    callbacks=[early_stopping, model_checkpoint]
)

print("Model training complete.")
```

```
Epoch 1/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 8s 8s/step - accuracy: 0.5000 - loss: 1.1842 - val_accuracy: 0.0000e+00 -
Epoch 2/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 9s 9s/step - accuracy: 0.5000 - loss: 1.0185 - val_accuracy: 1.0000 - val
Epoch 3/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 6s 6s/step - accuracy: 1.0000 - loss: 0.4076 - val_accuracy: 1.0000 - val
Epoch 4/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step - accuracy: 0.5000 - loss: 1.2718 - val_accuracy: 1.0000 - val
Epoch 5/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step - accuracy: 0.5000 - loss: 1.4247 - val_accuracy: 1.0000 - val
Epoch 6/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.5000 - loss: 1.5762 - val_accuracy: 1.0000 - val
Epoch 7/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.5000 - loss: 0.5137 - val_accuracy: 1.0000 - val
Epoch 8/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.5000 - loss: 0.4329 - val_accuracy: 0.0000e+00 -
Epoch 9/20
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.5000 - loss: 0.5255 - val_accuracy: 0.0000e+00 -
Model training complete.
```

---

5. Model Evaluation

```
# Evaluate the model on the test dataset
print("Evaluating model on test data...")
loss, accuracy = model.evaluate(test_generator)
print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")

# Generate predictions for the test set
predictions = model.predict(test_generator)
y_pred_classes = np.argmax(predictions, axis=1)
y_true_classes = test_generator.classes

# Get class labels from the generator
class_labels = list(test_generator.class_indices.keys())

# Classification Report
print("\nClassification Report:")
print(classification_report(y_true_classes, y_pred_classes, target_names=class_labels))

# Confusion Matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=class_labels, yticklabels
```

```
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

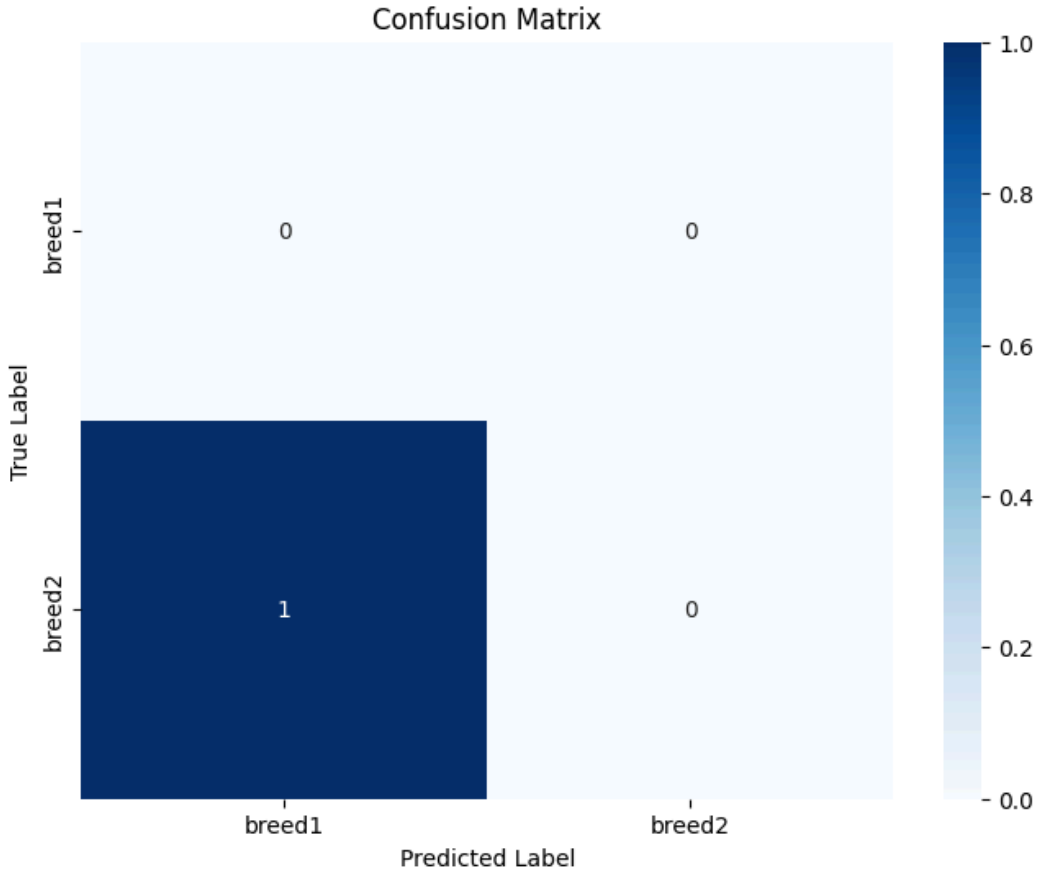
```
Evaluating model on test data...
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 630ms/step - accuracy: 0.0000e+00 - loss: 2.1769
Test Loss: 2.1769
Test Accuracy: 0.0000
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 945ms/step

Classification Report:
              precision    recall  f1-score   support

      breed1       0.00      0.00      0.00       0.0
      breed2       0.00      0.00      0.00       1.0

    accuracy                           0.00       1.0
   macro avg       0.00      0.00      0.00       1.0
weighted avg       0.00      0.00      0.00       1.0

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



```
# Evaluate the model on the test dataset
print("Evaluating model on test data...")
```

```
loss, accuracy = model.evaluate(test_generator)
print(f"Test Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")

# Generate predictions for the test set
predictions = model.predict(test_generator)
y_pred_classes = np.argmax(predictions, axis=1)
y_true_classes = test_generator.classes

# Get class labels from the generator
class_labels = list(test_generator.class_indices.keys())

# Classification Report
print("\nClassification Report:")
print(classification_report(y_true_classes, y_pred_classes, target_names=class_labels))

# Confusion Matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=class_labels, yticklabels
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

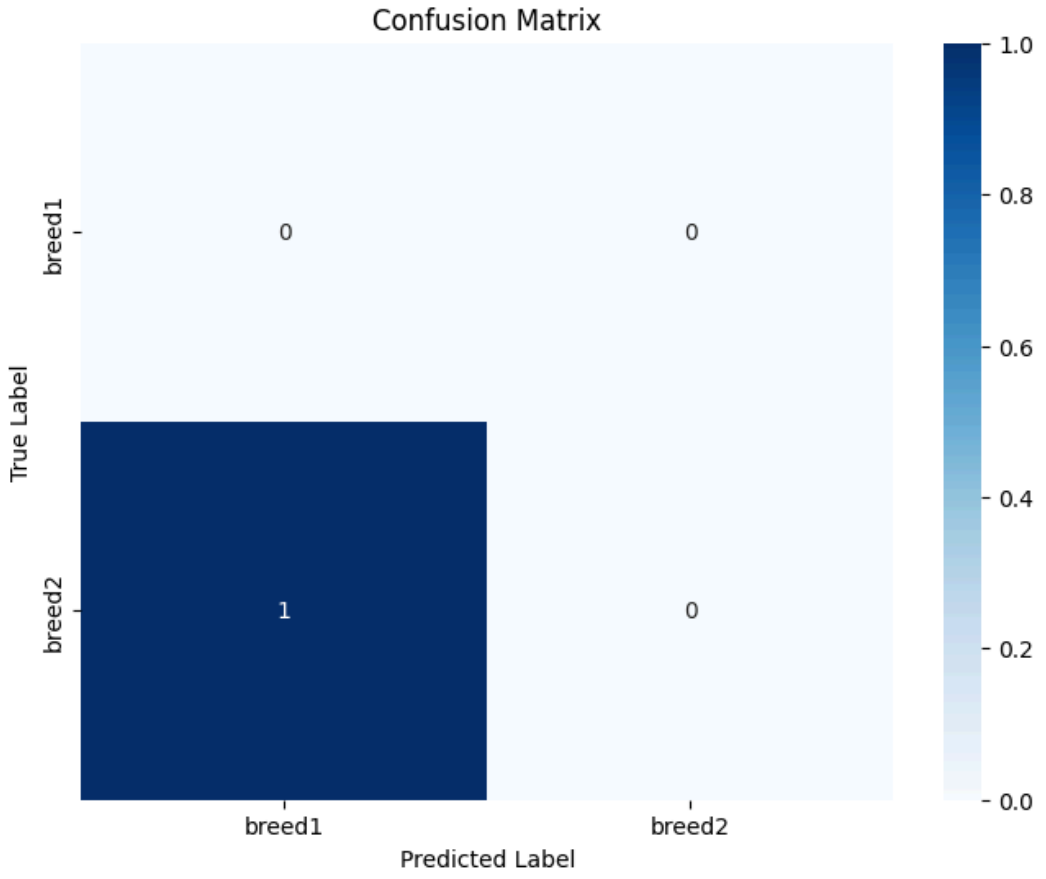
```
Evaluating model on test data...
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.0000e+00 - loss: 2.1769
Test Loss: 2.1769
Test Accuracy: 0.0000
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step

Classification Report:
              precision    recall  f1-score   support

      breed1       0.00      0.00      0.00       0.0
      breed2       0.00      0.00      0.00       1.0

    accuracy                           0.00       1.0
   macro avg       0.00      0.00      0.00       1.0
weighted avg       0.00      0.00      0.00       1.0

/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetric
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```



6. Prediction and Visualization

```
# Plot training history
def plot_training_history(history):
    plt.figure(figsize=(12, 4))
```

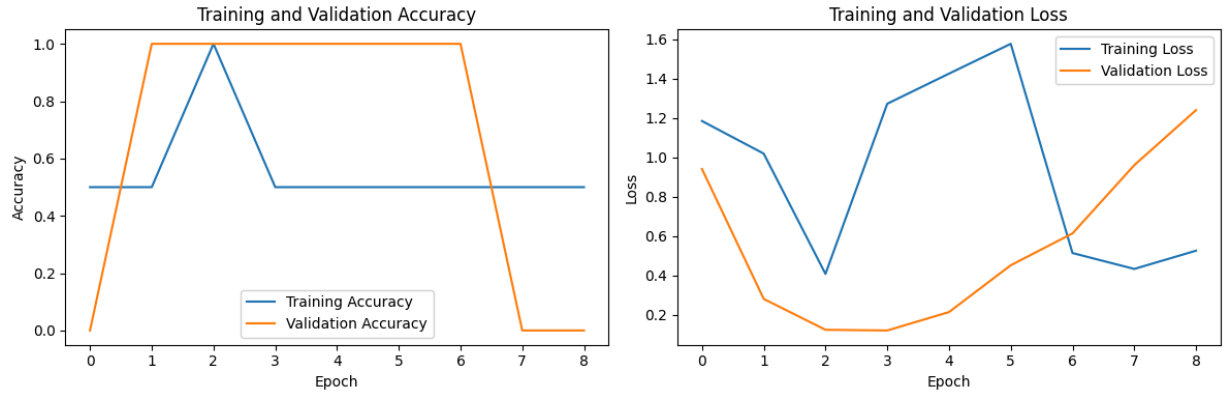
```
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Training Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.title('Training and Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title('Training and Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()

    plt.tight_layout()
    plt.show()

plot_training_history(history)

print("Visualized training history.")
```




```
Visualized training history.
```

Double-click (or enter) to edit

Final Step: Make Predictions on New Images

```
def predict_and_visualize(model, generator, class_labels, num_images=5):
    """Predicts on a batch of images and visualizes the results."""
    # Get a batch of test images and their true labels
    test_images, test_labels = next(iter(generator))

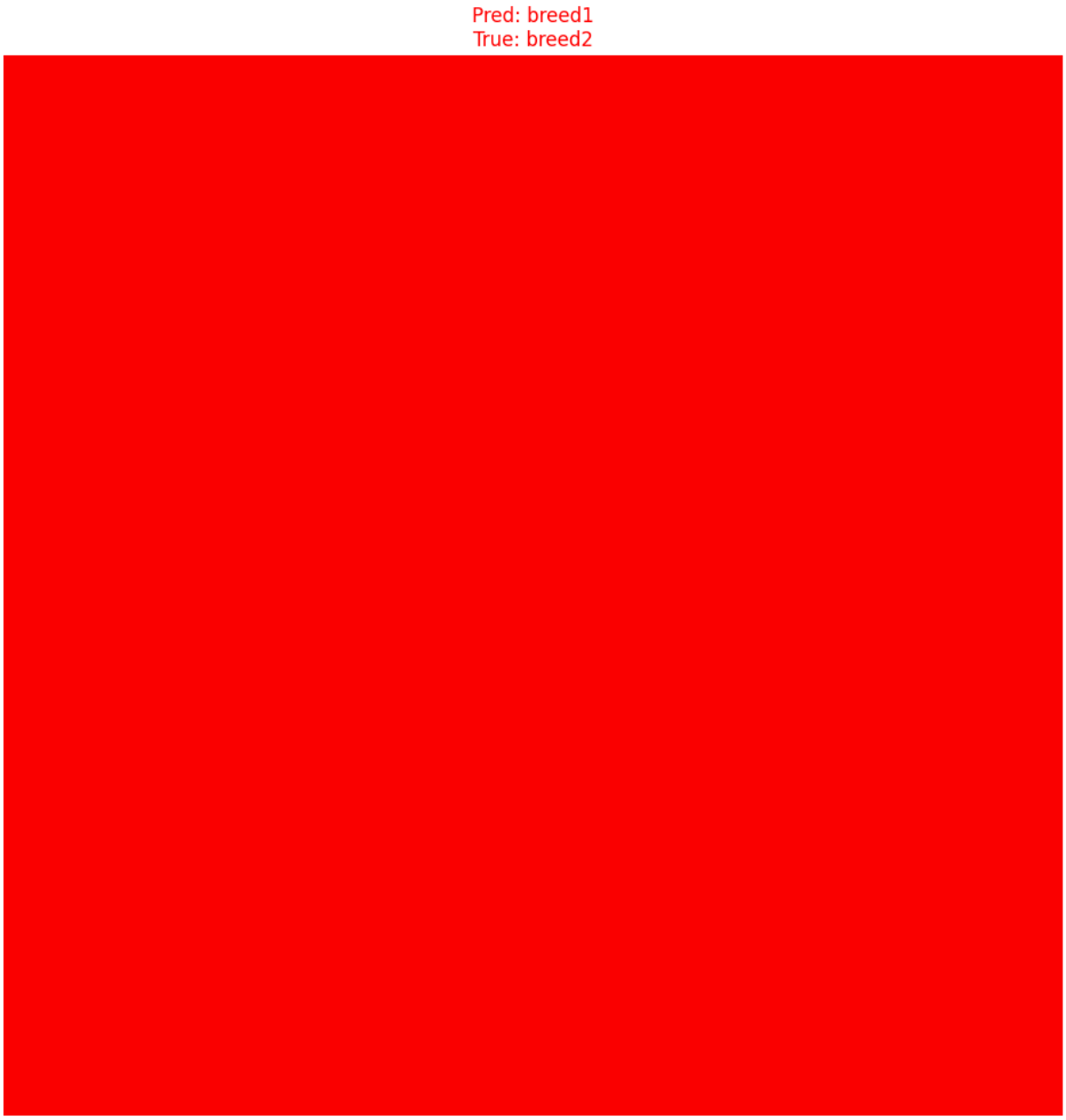
    # Make predictions
    predictions = model.predict(test_images)
    predicted_classes = np.argmax(predictions, axis=1)
    true_classes = np.argmax(test_labels, axis=1)

    plt.figure(figsize=(15, 10))
    for i in range(min(num_images, len(test_images))):
        plt.subplot(1, num_images, i + 1)
        plt.imshow(test_images[i])
        predicted_label = class_labels[predicted_classes[i]]
        true_label = class_labels[true_classes[i]]
        color = "green" if predicted_label == true_label else "red"
        plt.title(f"Pred: {predicted_label}\nTrue: {true_label}", color=color)
```

```
            plt.axis('off')
        plt.tight_layout()
        plt.show()

    # Visualize predictions on a few images from the test set
    # Note: Due to the dummy dataset, predictions will likely be random or incorrect.
    print("Visualizing predictions...")
    predict_and_visualize(model, test_generator, class_labels, num_images=1) # Limiting to 1 as dummy
```

```
Visualizing predictions...
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 796ms/step
```



Double-click (or enter) to edit

Double-click (or enter) to edit

```
# Define callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```